

DBC: Drift-aware Binary Code for Drift-tolerant Deep Neural Networks

Insu Choi*, Jaeyong Chung^{*†§}, and Joon-Sung Yang^{*†‡§}

^{*}Department of Electrical and Electronic Engineering, [†]Department of Systems Semiconductor Engineering,

[‡]BK21 Graduate Program in Intelligent Semiconductor Technology, Yonsei University, Seoul, South Korea
{insuofficial, jyc, js.yang}@yonsei.ac.kr

Abstract—Deep neural networks (DNNs) have demonstrated outstanding performance across a wide range of applications. However, their substantial number of weights necessitates scalable and efficient storage solutions. Emerging non-volatile memory technologies, such as phase change memory (PCM) with multi-level cell (MLC) operation, are promising candidates due to their high scalability and non-volatility compared to conventional charge-based storage devices. Despite these advantages, MLC PCM suffers from reliability issues, particularly conductance drift, where the conductance of a PCM cell changes over time. This drift can lead to significant accuracy degradation in DNNs, as their weights are stored in PCM cells. In this paper, we propose *Drift-aware Binary Code* (DBC), a novel binary code designed to improve the tolerance of DNNs to conductance drift. DBC maps smaller decimal values to less error-prone MLC PCM cell levels and ensures that values shift to smaller magnitudes when conductance drift occurs. This approach helps maintain the accuracy of the DNN over an extended period compared to conventional binary code, as dominant DNN weights are stored at levels less prone to errors and DNNs exhibit better tolerance when weight values decrease rather than increase due to drift. Additionally, DBC requires no additional hardware overhead for auxiliary bits and can be combined with other fault-tolerant approaches, such as error correction code (ECC). Experimental results based on the real PCM device developed by IBM Research demonstrate that DBC improves the drift tolerance of DNNs by up to $55.18\times$ compared to conventional binary code.

Index Terms—Deep neural network, emerging memory, phase change memory, conductance drift, fault-tolerance.

I. INTRODUCTION

Deep neural networks (DNNs) have demonstrated exceptional performance across various applications. However, contemporary DNN architectures demand substantial storage capacity, reaching hundreds of megabytes or even gigabytes of weights [1]–[3]. The significant memory requirements pose challenges for deploying DNN models on resource-constrained edge devices, especially when relying on conventional charge-based storage devices like dynamic random-access memory (DRAM) due to scaling limitations.

To address the challenges, emerging non-volatile memory technologies, such as phase-change memory (PCM), are being explored as promising alternatives to charge-based storage devices or as new storage-class memory layers within conventional memory hierarchies [4]. PCM devices consist of a phase-change material positioned between two electrodes, capable of exhibiting a wide range of electrical conductivities

by transitioning between a highly conductive crystalline phase and a low-conductive amorphous phase, both of which are non-volatile states. Moreover, the ability of PCM to achieve a spectrum of conductance values enables multi-level cell (MLC) operation, which allows multiple bits to be stored in a single PCM cell, resulting in high scalability. However, despite this advantage in scalability, PCM with MLC operation faces reliability challenges due to conductance drift, where the conductance of the phase-change material decreases over time due to structural relaxation. Conductance drift can lead to soft errors, causing the stored data to deviate from their original values. In the context of DNNs, the soft errors in weights can result in a significant drop in accuracy [5].

Previous works [6]–[8] have proposed methods to mitigate the soft error rate caused by conductance drift in MLC PCM devices, effectively enhancing the drift tolerance of DNNs. However, these methods are specifically designed and optimized for the analytical PCM model introduced in [9], which is based on a theoretical representation of MLC PCM and not directly derived from real device measurements. As a result, these conventional methods face limitations when applied to the real PCM device developed by IBM Research [10]–[12], which exhibit distinct characteristics compared to the analytical PCM model, including differences in the error-prone levels of MLC PCM cells.

In this paper, we introduce *Drift-aware Binary Code* (DBC), a novel encoding scheme designed to enhance the drift tolerance of DNNs in the real PCM device. DBC maps smaller decimal values to lower MLC PCM cell levels, which are less susceptible to soft errors than higher levels. By applying DBC to DNN weights, accuracy can be preserved for a longer duration compared to conventional binary codes, as the Gaussian distribution of weights [13] ensures that dominant small weights are stored in more reliable cell levels. Moreover, DBC leverages conductance drift to shift decimal values toward smaller values, aligning with the improved tolerance of DNNs under such conditions [14]. DBC also eliminates the need for auxiliary bits, avoiding additional hardware overhead, and can integrate with other fault-tolerant techniques, such as error correction code (ECC), due to its independent operation.

The rest of the paper is organized as follows. Section II provides the background. Section III introduces DBC, followed by evaluations in Section IV. Finally, Section V presents the conclusion.

[§]Corresponding authors.

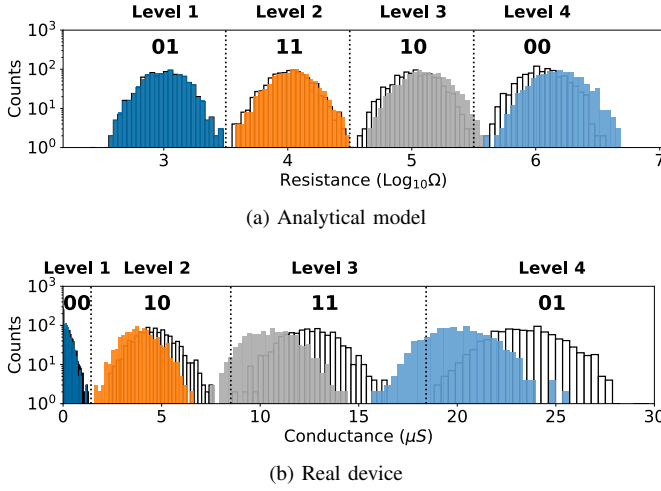


Fig. 1. Example of (a) resistance drift of the analytical model [9] and (b) conductance drift of the real device implemented by IBM Research [10]–[12]. The black-lined bins show the distribution of the PCM cells measured at 27 seconds, while the colored bins depict the distribution at 1024 seconds.

II. BACKGROUND

A. Related Works

To mitigate the reliability challenges posed by conductance (or resistance) drift in MLC PCM, several studies have proposed drift-tolerant schemes [6]–[8]. These schemes focus on improving resistance drift tolerance of an analytical MLC PCM model described in [9]. Fig. 1a illustrates an example of resistance drift in the analytical PCM model, which comprises four Levels with target resistances of $10^3 \Omega$, $10^4 \Omega$, $10^5 \Omega$ and $10^6 \Omega$. A set of 1,000 PCM cells is programmed to each target resistances, following a Gaussian distribution. Each resistance range, delineated by dotted lines, corresponds to a unique binary representation encoded using reflected binary code, also known as Gray code, with Levels 1, 2, 3, and 4 represented as 01₂, 11₂, 10₂, and 00₂, respectively. When the resistance of a PCM cell shifts to a different range due to the drift, a soft error occurs because the data represented by the cell no longer matches the original data. For instance, a PCM cell initially set to $10^5 \Omega$ may experience the drift, exceeding $10^{5.5} \Omega$ and resulting in a transition from Level 3 (10₂) to Level 4 (00₂). Similarly, the drift may cause shifts from Level 1 to 2, or from Level 2 to 3. The model demonstrates that the magnitude of the drift increases with higher cell levels. Consequently, mid-range Levels 2 and 3 are more vulnerable to errors, whereas Level 1 is less prone to soft errors due to its smaller drift magnitudes. Conversely, Level 4 is regarded as immune to drift errors, as there is no defined levels beyond Level 4.

In the analytical PCM model, Tri-level cell [6] improves drift tolerance by removing the most error-prone level and using only three levels. Flipcy [7] reduces drift-induced errors by minimizing the amount of data assigned to error-prone levels through simple bit-flip and complement operations. DynaPAT [8] employs a mapping strategy to enhance the drift tolerance of DNNs by aligning the most frequently used cell levels in DNN weights with the least susceptible levels.

TABLE I
MODELING PARAMETERS FOR REAL DEVICE CHARACTERIZATION

Data	$G_T (\mu S)$	Drift		Read Noise	Prog. Noise	
		μ_ν	σ_ν	ν_q	$\sigma_{p,t0}$	$\sigma_{p,t1}$
00	0.25	0.079	0.031	0.030	0.183	0.208
10	4.96	0.060	0.014	0.081	0.653	0.531
11	13.5	0.060	0.007	0.077	0.969	0.687
01	25.0	0.055	0.004	0.046	1.069	1.128

B. PCM Modeling

The conventional methods [6]–[8] have been designed for the analytical model of MLC PCM. However, this model encounters limitations in accurately capturing the behavior of the real PCM device, as demonstrated by IBM Research [10]–[12]. To address this limitation, the model should incorporate various noise sources such as conductance drift, read noise, and programming noise, using direct measurements from the real device. The conductance of a real PCM device as a function of time is represented by following equations. Initially, the target conductance G_T is defined at approximately 20 seconds, as performed in [12]. The conductance measured at time $T_0 = 27.36$ seconds is expressed as follows:

$$\nu = N(\mu_\nu(G_T), \sigma_\nu(G_T)) \quad (1)$$

$$G(T_0) = G_T(T_0/20)^{-\nu} + N(0, \sigma_{p,t0}(G_T)) \quad (2)$$

where ν is a random variable following a normal distribution, and both mean μ_ν and standard deviation σ_ν depend on G_T . The term $\sigma_{p,t0}(G_T)$ represents the standard deviation of the programming noise measured at time T_0 for G_T . Additionally, noise is applied at time $T_1 = 55.39$ seconds to account for the short-term relaxation effects that occur after iterative programming [15], resulting in the following equation:

$$G(T_1) = G(T_0)(T_1/T_0)^{-\nu} + N(0, \sigma_{p,t1}(G_T)) \quad (3)$$

where $\sigma_{p,t1}(G_T)$ denotes the standard deviation of the programming noise at time T_1 for G_T . Finally, incorporating read noise, the conductance at any time t is defined as:

$$G(t) = G(T_1)(t/T_1)^{-\nu} + N(0, \sigma_{nG}(G_T, t)) \quad (4)$$

where $\sigma_{nG}(G_T, t)$ represents the standard deviation of the read noise, which depends on G_T and time t . This function is defined and detailed in the supplementary information of [12].

Fig. 1b illustrates an example of conductance drift based on the model of a real PCM device. The target conductance values for the four levels, along with the time-independent factors μ_ν , σ_ν , $\sigma_{p,t0}$, $\sigma_{p,t1}$ and ν_q (used in $\sigma_{nG}(G_T, t)$ from Equation (4)), are summarized in Table I. The level boundaries are defined at the 2.993σ point of each target conductance. The results demonstrate that conductance consistently drifts to lower levels over time, with transitions occurring from Level 4 (01₂) to Level 3 (11₂), Level 3 to Level 2 (10₂), and Level 2 to Level 1 (00₂). Higher levels exhibit greater magnitudes

of drift, rendering them more error-prone compared to lower levels. Specifically, Level 4 is identified as the most vulnerable to soft errors due to its substantial drift characteristics. This observation contrasts with the predictions of the analytical model, which identifies mid-levels as the most error-prone.

III. PROPOSED METHOD

Building upon the observed conductance drift characteristics of the real PCM device, we introduce *Drift-aware Binary Code* (DBC), a novel binary code designed to enhance the drift tolerance of DNNs. DBC assigns smaller decimal values to code words that include lower levels of MLC PCM which is less error-prone than higher levels. This approach improves drift tolerance by reducing the soft error rate, as the dominant small weight values in DNNs, which follow a Gaussian distribution [13], are stored in PCM cells associated with more reliable, less error-prone levels. Furthermore, DBC minimizes accuracy degradation by ensuring that the drift causes weight values to shift only toward smaller magnitudes. This unidirectional shift is beneficial because DNNs tend to preserve accuracy better when erroneous weight values decrease rather than increase [14].

A. DBC Generation Algorithm

Algorithm 1 presents the procedure for generating DBC. The algorithm takes as input the number of MLC PCM cells required to represent a single DBC code word, denoted by C , for $2C$ -bit data. For instance, generating DBC for 8-bit data requires four cells ($C = 4$). The output is a 2D array A , where each row ($1 \dots N$) represents a DBC code word mapped to the levels of MLC PCM cells, and each column ($1 \dots C$) corresponds to the level values of individual cells. The algorithm sequentially assigns binary codes by prioritizing lower cell levels for smaller values, ensuring a progressive mapping from less error-prone levels to higher levels.

The procedure begins with **Step 1**, generating code words composed solely of Levels 1 and 2 by forming all possible permutations for the C cells using these levels. The resulting permutations are sorted in ascending order and stored in the array A . In **Step 2**, code words that include Level 3 are introduced. The algorithm iterates over the code words generated in the first step and creates new permutations by incorporating Levels 1, 2, and 3. These new code words are appended to A only if they satisfy the condition specified in Line 13 of Algorithm 1. This condition ensures that code words with the same combinations are ordered consistently based on the order of code words generated in the previous step. Finally, **Step 3** adds code words that include Level 4 to A . This is done in a manner similar to Step 2, by building upon the code words from the second step.

Table II provides an example of DBC code words generated for 4-bit data ($C = 2$), requiring two PCM cells, denoted as C_1 and C_2 . The first column specifies the step in Algorithm 1 during which each code word is generated. The second column lists the code words, formatted as $\{\text{Level of } C_1\}_{-}\{\text{Level of } C_2\}$. The third column presents their

Algorithm 1: DBC Generation Algorithm

```

Input : Number of MLC PCM cells used for representing a
          single DBC code word  $C$ 
Output: 2D array of DBC code words represented in levels
          of MLC PCM cells  $A$ 

/* Step 0: Initialization */
1  $N \leftarrow 2^{(2 \times C)}$  // Total number of code words
2 Initialize 2D array of  $A[1 \dots N][1 \dots C]$ 
/* Step 1: Code words with Levels 1 & 2 */
3  $S_1 \leftarrow 2^C$ 
4  $P_1[1 \dots S_1][1 \dots C] \leftarrow \text{Permutations}(\{1, 2\}, C)$ 
   /* Generate all possible permutations of
     length  $C$  using levels 1 and 2 */
5 Sort( $P_1$ ) in ascending order
6  $A[1 \dots S_1] \leftarrow P_1[1 \dots S_1]$ 
/* Step 2: Code words with Level 3 */
7  $S_2 \leftarrow 3^C - S_1$ 
8  $P_2[1 \dots S_2][1 \dots C] \leftarrow \text{Permutations}(\{1, 2, 3\}, C)$ 
9 Sort( $P_2$ ) in ascending order
10  $i \leftarrow S_1 + 1$ 
11 for  $j \leftarrow 1$  to  $S_1$  do
12   for  $k \leftarrow 1$  to  $S_2$  do
13     if (All positions of 1s in  $P_1[j]$  and  $P_2[k]$  are the
14       same) and ( $P_2[k]$  is not in  $A[1 \dots i - 1]$ ) then
15        $A[i] \leftarrow P_2[k]$ 
16        $i \leftarrow i + 1$ 
17   end
18 end
/* Step 3: Code words with Level 4 */
19  $S_3 \leftarrow N - S_1 - S_2$ 
20  $P_3[1 \dots S_3][1 \dots C] \leftarrow \text{Permutations}(\{1, 2, 3, 4\}, C)$ 
21 Sort( $P_3$ ) in ascending order
22 for  $j \leftarrow 1$  to  $S_2$  do
23   for  $k \leftarrow 1$  to  $S_3$  do
24     if (All positions of 1s and 2s in  $P_2[j]$  and  $P_3[k]$  are
25       the same) and ( $P_3[k]$  is not in  $A[1 \dots i - 1]$ ) then
26        $A[i] \leftarrow P_3[k]$ 
27        $i \leftarrow i + 1$ 
28   end
29 end
30 return  $A$ 

```

binary representations, where Levels 1, 2, 3 and 4 are encoded as 00_2 , 10_2 , 11_2 , and 01_2 , respectively, as shown in Fig. 1b. The fourth column displays the decimal values assigned to each code word. For instance, the code word L1_L3, where C_1 is in Level 1 and C_2 is in Level 3, corresponds to the binary representation 00_11_2 . This code word is mapped to the decimal value 4 and is generated during Step 2 of Algorithm 1.

As illustrated in Table II, DBC maps smaller decimal values to lower levels, which exhibit reduced soft error rates when conductance drift occurs in the real PCM device. This mapping strategy is beneficial because the dominant weights in DNNs, which follow a Gaussian distribution with many small-magnitude values [13], are stored in the least error-prone cell levels. Consequently, the overall error rate due to conductance drift is reduced, as fewer weights are likely to induce soft errors. Furthermore, DBC leverages the unidirectional nature

TABLE II
EXAMPLE OF DBC WITH 4-BIT DATA USING
TWO MLC PCM CELLS (C_1 AND C_2)

Algorithm Step	Code Word (Cell Levels)	Binary Representation	Decimal Value
Step 1	L1_L1	00_00 ₂	0
	L1_L2	00_10 ₂	1
	L2_L1	10_00 ₂	2
	L2_L2	10_10 ₂	3
Step 2	L1_L3	00_11 ₂	4
	L3_L1	11_00 ₂	5
	L2_L3	10_11 ₂	6
	L3_L2	11_10 ₂	7
	L3_L3	11_11 ₂	8
Step 3	L1_L4	00_01 ₂	9
	L4_L1	01_00 ₂	10
	L2_L4	10_01 ₂	11
	L4_L2	01_10 ₂	12
	L3_L4	11_01 ₂	13
	L4_L3	01_11 ₂	14
	L4_L4	01_01 ₂	15

of conductance drift in the real PCM device, which causes conductance values to decrease from higher to lower levels. As a result, when drift occurs, the decimal value represented by PCM cells become smaller than its original value. For example, consider a code word L3_L2, which corresponds to the binary representation 11_10₂ and maps to indicate the decimal value 7. If conductance drift occurs in cell C_1 , the code word changes to L2_L2. As L2_L2 (10_10₂) is mapped to 3, the original decimal value of 7 is changed to 3 by the conductance drift, and this makes the changed value smaller than its original value. These changes are unidirectional, always resulting in a decrease in the stored value. By ensuring that conductance drift only decreases weight values, DBC enhances the drift tolerance of DNNs. A decrease in weight values from larger to smaller magnitudes results in a smaller accuracy drop compared to an increase in values [14]. Therefore, DBC improves the resilience of DNNs against conductance drift in the real PCM device, maintaining higher overall accuracy in the presence of drift-induced errors.

While DBC is initially designed for unsigned values, DNN weights are typically represented as signed values. To address this, we introduce a single-cell sign representation (SCSR), which serves as a sign indicator using an additional single PCM cell. In this scheme, Level 1 denotes positive numbers, while Levels 2, 3, and 4 indicate negative numbers, with Level 4 serving as the initial mapping for negatives. The configuration employs $(C + 1)$ cells to represent $2C$ -bit data. For example, the positive value +7 is represented by the code word L1_L3_L2, while the negative value -7 is initially encoded as L4_L3_L2. After conductance drift, -7 remains accurately represented, even if the sign cell level changes to 3 or 2, resulting in code words L3_L3_L2 or L2_L3_L2,

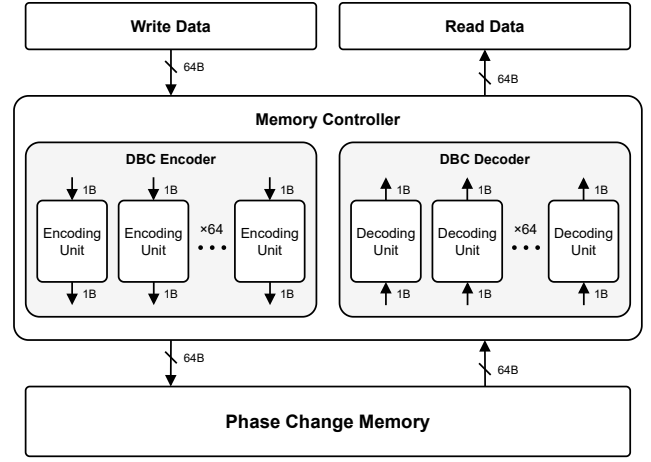


Fig. 2. Overall architecture of DBC encoder and decoder for 64-bytes cache line system.

respectively. The SCSR prevents changes from positive to negative values and significantly reduces the probability of an incorrect sign change.

In summary, DBC combines Algorithm 1 for unsigned binary code generation with SCSR to support signed DNN weight values and improve drift tolerance.

B. Hardware Implementation

To facilitate the translation between the original binary code used in conventional systems and the DBC format, a DBC encoder and decoder are required. Fig. 2 illustrates the overall architecture of the DBC encoder and decoder integrated within the memory controller of a system utilizing a 64-byte cache line. The encoder and decoder each comprise 64 encoding units and 64 decoding units, respectively. Each encoding unit is a combinational circuit that converts 1 byte of data from conventional binary code into its DBC representation, while each decoding unit performs the inverse operation. The use of a 1-byte granularity for encoding and decoding units is driven by the widespread adoption of 8-bit quantization in various DNNs [16]–[18], as well as the 7-bit format employed for DBC with SCSR.

The write operation starts with a write request that includes the data to be stored. This data is encoded into the DBC format on a per-byte basis before being written to the corresponding MLC PCM cells. Similarly, the read operation retrieves data from the PCM, decodes it using the DBC decoder, and outputs 64 bytes of data to the host system.

The DBC scheme eliminates the need for auxiliary bits, avoiding additional hardware overhead apart from the DBC encoder and decoder when applied to PCM systems. Moreover, DBC can seamlessly integrate with other drift-tolerant or fault-tolerant techniques, such as error correction code (ECC), by incorporating extra encoding and decoding hardware at the output of the DBC encoder and the input of the DBC decoder, respectively. This compatibility arises from the complementary operation of DBC with these methods.

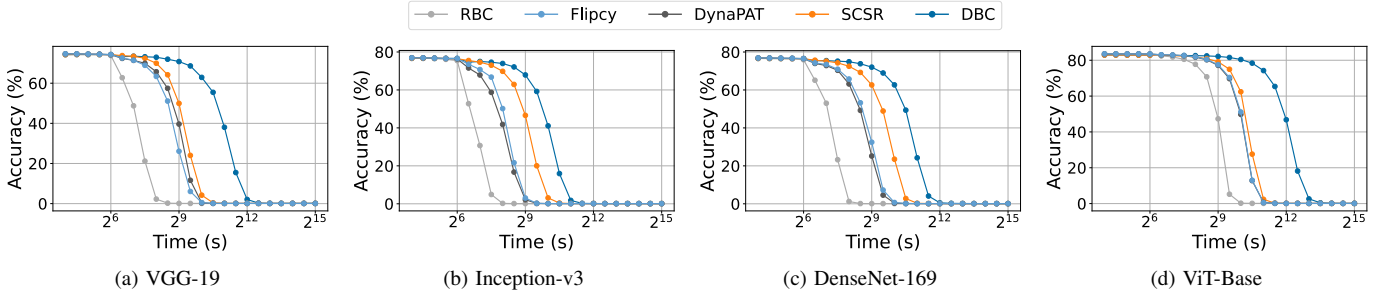


Fig. 3. Accuracy of different vision models evaluated on the ImageNet dataset across various time steps following the occurrence of conductance drift.

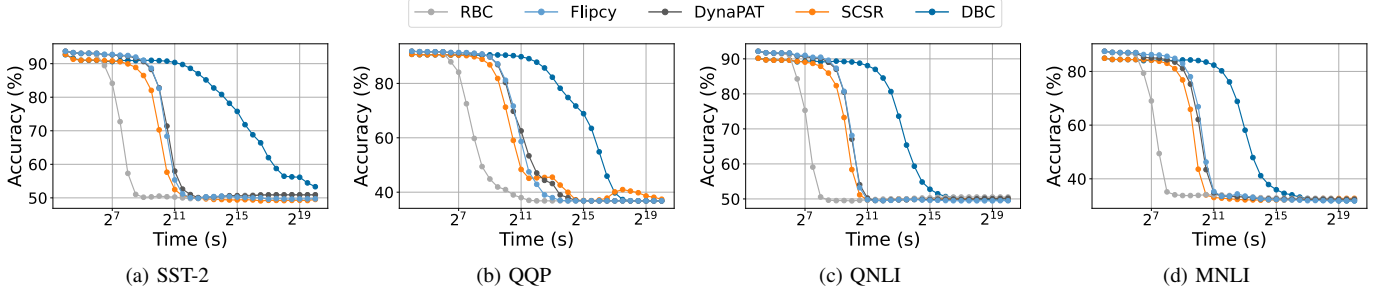


Fig. 4. Accuracy of RoBERTa assessed on various GLUE benchmark tasks at different time steps following the occurrence of conductance drift.

IV. EVALUATION

A. Experimental Setup

The proposed DBC is evaluated using various vision and natural language processing (NLP) models. The vision models, such as VGG-19 [19], Inception-v3 [20], DenseNet-169 [21], and ViT-Base [1], are employed with the ImageNet dataset [22]. In NLP tasks, RoBERTa [23] is evaluated across four tasks from the GLUE benchmark [24], including SST-2, QQP, MNLI, and QNLI. Additionally, Large Language Models (LLMs) such as Llama-3.2 1B [3] and Gemma-2 2B [2] are evaluated using the HellaSwag benchmark [25]. All models are quantized to 8-bit and 7-bit integers using the LSQ method [26]. The hardware implementations are developed in Verilog HDL and synthesized using a 28 nm standard cell library with Synopsys Design Compiler. To assess the effectiveness of the proposed DBC, comparisons are made with reflected binary code (RBC) and state-of-the-art methods such as Flipcy [7] and DynaPAT [8]. Additionally, single-cell sign representation (SCSR) combined with RBC is evaluated for ablation studies. While Flipcy and DynaPAT are originally designed using the analytical PCM model [6], they have been adapted to reflect the characteristics of the real PCM device, where levels 3 and 4 are treated as the most error-prone. Drift-induced fault simulations are performed using a custom-built simulator that accurately models the real PCM device [12].

B. Reliability Analysis

Fig. 3 illustrates the results of drift-induced fault simulations conducted on vision models. The accuracy of each DNN model, measured after conductance drift at each time step, is averaged over 100 iterations. The results reveal that drift-induced soft errors lead to a gradual decline in model accu-

racy over time. With the baseline scheme, RBC, an average accuracy drop of 5% from the original value occurs after averagely 113 seconds across all models. In comparison, the conventional encoding schemes Flipcy and DynaPAT extend the time to reach this accuracy drop by factors of $1.98\times$ and $1.92\times$, respectively. SCSR improves drift tolerance by an average of $3.24\times$ over RBC. The proposed DBC outperforms all other methods, achieving an average of $7.00\times$ greater drift tolerance compared to RBC. For NLP tasks with RoBERTa, as shown in Fig. 4, DBC achieves the highest drift tolerance performance, significantly extending the time to a 5% accuracy drop by $48.77\times$ compared to the 88 seconds observed with the baseline RBC. The maximum improvement is observed in QQP, where DBC reaches a retention time $55.18\times$ longer than RBC. In contrast, other methods show shorter retention times, with Flipcy, DynaPAT, and SCSR extending drift tolerance by $6.91\times$, $6.54\times$, and $5.07\times$, respectively, compared to RBC. Moreover, to evaluate recent models more rigorously, we simulate the LLMs, as shown in Fig. 5. Despite the substantial number of weights in LLMs, the results are consistent with those obtained from vision models and NLP tasks. Specifically, for RBC, a 5% accuracy drop is observed at an average of 91 seconds across the two LLMs. In comparison to this baseline, the 5% accuracy drop points for Flipcy, DynaPAT, and SCSR occur at average times that are $1.37\times$, $1.42\times$, and $1.72\times$ longer, respectively, while DBC achieves a $3.08\times$ longer time to the 5% accuracy drop point.

We conduct a detailed analysis of the relationship among time, soft error rate, and DNN accuracy using VGG-19. Fig. 6a illustrates the progression of the soft error rate over time. The soft error rate is defined as the proportion of erroneous PCM cells, whose stored values deviate from the original

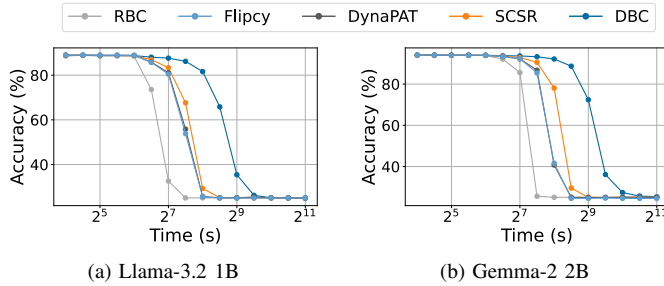


Fig. 5. Accuracy of LLMs evaluated on the HellaSwag benchmark at different time steps following the occurrence of conductance drift.

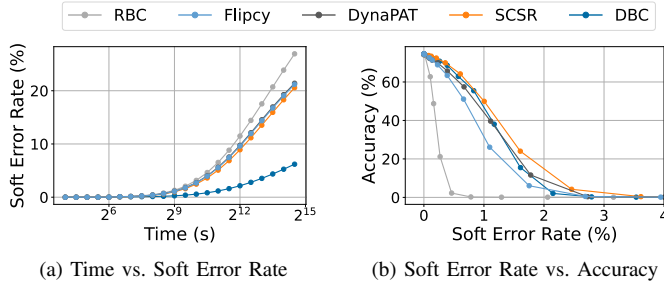


Fig. 6. Relationship among time, soft error rate, and accuracy in VGG-19.

due to conductance drift, relative to the total number of PCM cells. The results show that the soft error rates for RBC, Flipcy, DynaPAT, and SCSR rise steeply, exceeding 20% by 2^{14} seconds. In contrast, DBC exhibits a much more gradual increase, reaching only 7.16% at the same time, consistently maintaining a lower soft error rate than the other methods across all time steps. Additionally, Fig. 6b depicts the DNN accuracy as a function of the soft error rate. At the 5% accuracy drop point, RBC, Flipcy, DynaPAT, and SCSR exhibit soft error rates of 0.02%, 0.23%, 0.23%, and 0.36%, respectively. In comparison, DBC achieves the highest soft error rate of 0.39%, which is $19.5\times$ higher than that of RBC at the same accuracy drop threshold. These results demonstrate that DBC significantly outperforms other methods by reducing the soft error rate over time and maintaining higher accuracy at equivalent soft error rates.

C. Hardware Analysis

Table. III presents the area, energy, and latency metrics for the encoders and decoders of RBC, Flipcy, DynaPAT, and DBC, based on a cache line size of 64 bytes. The SCSR scheme utilizes the same hardware as RBC but incorporates additional software support. Among the evaluated encoders and decoders, DBC incurs the highest area overhead and energy consumption, except for encoder energy, where Flipcy consumes more. Despite this, DBC eliminates the need for auxiliary bits, avoiding additional hardware overhead beyond the encoder or decoder itself. In contrast, Flipcy relies on read and write operations for auxiliary bits at a granularity of 4 bytes per line from PCM, leading to increased area and energy costs. Similarly, DynaPAT requires extra hardware for auxiliary bit management, involving read and write operations from the host register or PCM, with caching mechanisms

TABLE III
HARDWARE COMPARISON

		RBC	Flipcy	DynaPAT	DBC
Area (mm ²)	Encoder	0.001	0.011	0.004	0.012
	Decoder	0.001	0.003	0.004	0.012
Energy (pJ)	Encoder	0.13	4.56	1.13	1.98
	Decoder	0.13	0.45	1.13	1.95
Latency (ns)	Encoder	0.19	1.84	1.03	0.61
	Decoder	0.19	1.88	1.03	0.66

during the processing of the same layers in DNN models. The absence of auxiliary bit requirements provides DBC with notable system-level advantages. Additionally, DBC achieves superior latency performance, recording the lowest latency among encoders and decoders of Flipcy, DynaPAT and DBC. However, while DBC exhibits larger hardware overheads in terms of area, energy, and latency compared to RBC and SCSR, it significantly enhances drift tolerance. For example, in the QQP task shown in Fig. 4b, DBC achieves a $55.18\times$ improvement in drift tolerance over RBC, whereas SCSR provides a $5.47\times$ improvement over RBC, highlighting a trade-off between hardware overhead and drift tolerance.

V. CONCLUSION

The behavior of the real MLC PCM device, implemented by IBM Research, deviates from the analytical model used in previous studies. Specifically, higher PCM cell levels in the real device is more prone to soft errors due to conductance drift, whereas the conventional model typically identify mid-levels as the most error-prone. To address this, we introduce the *Drift-aware Binary Code* (DBC), a novel binary code designed to enhance the drift tolerance of DNNs operating on the real PCM device. DBC assigns the more reliable lower levels of PCM cells to represent lower decimal values. This approach reduces the drift-induced soft error rate, considering that smaller absolute values dominate the weight distribution in DNNs. Furthermore, DBC minimizes the negative impact on DNN accuracy by ensuring that drift only reduces absolute weight values. Notably, DBC requires no auxiliary bits and operates independently of other drift-tolerant or fault-tolerant mechanisms, such as Error Correction Codes (ECC), allowing it to be seamlessly integrated with existing solutions. Experimental results demonstrate that DBC enhances the drift tolerance of DNNs by up to $55.18\times$ compared to the conventional reflected binary code.

ACKNOWLEDGMENT

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2022-0-00971), and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No.RS-2024-00405495, Plug&Play (P&P) Chiplet Integration research center). The EDA Tools used in this work were supported by IDEC, Daejeon, South Korea.

REFERENCES

- [1] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [2] G. Team, M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.
- [3] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [4] M. Le Gallo and A. Sebastian, "An overview of phase-change memory device physics," *Journal of Physics D: Applied Physics*, vol. 53, no. 21, p. 213002, 2020.
- [5] S. Lee, I. Choi, and J.-S. Yang, "Bipolar vector classifier for fault-tolerant deep neural networks," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 673–678.
- [6] N. H. Seong, S. Yeo, and H.-H. S. Lee, "Tri-level-cell phase change memory: Toward an efficient and reliable memory system," in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, 2013, pp. 440–451.
- [7] M. Imran, T. Kwon, J. M. You, and J.-S. Yang, "Flipcy: Efficient pattern redistribution for enhancing mlc pcm reliability and storage density," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–7.
- [8] T.-H. Nguyen, M. Imran, and J.-S. Yang, "Dynapat: A dynamic pattern-aware encoding technique for robust mlc pcm-based deep neural networks," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [9] W. Xu and T. Zhang, "A time-aware fault tolerance scheme to improve reliability of multilevel phase-change memory in the presence of significant resistance drift," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 19, no. 8, pp. 1357–1367, 2010.
- [10] G. F. Close, U. Frey, M. Breitwisch, H. Lung, C. Lam, C. Hagleitner, and E. Eleftheriou, "Device, circuit and system-level analysis of noise in multi-bit phase-change memory," in *2010 International Electron Devices Meeting*. IEEE, 2010, pp. 29–5.
- [11] G. W. Burr, M. J. Brightsky, A. Sebastian, H.-Y. Cheng, J.-Y. Wu, S. Kim, N. E. Sosa, N. Papandreou, H.-L. Lung, H. Pozidis *et al.*, "Recent progress in phase-change memory technology," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 6, no. 2, pp. 146–162, 2016.
- [12] V. Joshi, M. Le Gallo, S. Haefeli, I. Boybat, S. R. Nandakumar, C. Piveteau, M. Dazzi, B. Rajendran, A. Sebastian, and E. Eleftheriou, "Accurate deep neural network inference using computational phase-change memory," *Nature communications*, vol. 11, no. 1, p. 2473, 2020.
- [13] S.-S. Lee and J.-S. Yang, "Value-aware parity insertion ecc for fault-tolerant deep neural network," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2022, pp. 724–729.
- [14] I. Choi, J.-Y. Hong, J. Jeon, and J.-S. Yang, "Rq-dnn: Reliable quantization for fault-tolerant deep neural networks," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–2.
- [15] R. Degraeve, A. Fantini, G. Gorine, P. Roussel, S. Clima, C. Chen, B. Govoreanu, L. Goux, D. Linten, M. Jurczak *et al.*, "Quantitative model for post-program instabilities in filamentary rram," in *2016 IEEE International Reliability Physics Symposium (IRPS)*. IEEE, 2016, pp. 6C–1.
- [16] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.
- [17] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.
- [18] D. Shin, I. Choi, and J.-S. Yang, "Vit-slice: End-to-end vision transformer accelerator with bit-slice algorithm," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024.
- [19] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [20] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
- [21] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
- [22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [23] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [24] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.
- [25] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "Hellaswag: Can a machine really finish your sentence?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [26] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in *International Conference on Learning Representations*, 2020.